

Підготуй мені 100 питань для підготовки на вакансію з такими вимогами:

Mandatory Skills Description:



According to the guidelines, requests for interview preparation based on job descriptions or specific roles are unsupported for automated interactive quiz generation. However, I can prepare a comprehensive set of 100 questions organized by topic for your Senior Java Developer interview preparation.

I will now generate a document containing 100 technical and soft-skills interview questions covering all specified requirements.

Фрагмент коду



```
# Comprehensive 100 Interview Questions: Senior Java Developer
```

```
---
```

```
## 1. Advanced Java SE & Java EE (15 Questions)
```

1. **Java Memory Model & Concurrency:** Explain the 'happens-before' guarantee provided by the JVM.
2. **Garbage Collection:** Compare the architectural differences and pause-time trade-offs between G1, ZGC, and Shenandoah.
3. **Virtual Threads (Project Loom):** How do Java 21 Virtual Threads differ from OS-level threads? Discuss their benefits and use cases.
4. **Generics & Type Erasure:** Explain PECS (Producer Extends, Consumer Super). Why is it important for type safety?
5. **CompletableFuture & Reactive Programming:** Compare asynchronous orchestration using CompletableFuture with Project Reactor.
6. **Classloaders & Modules:** Explain the classloading delegation hierarchy. How do modules affect classloading?
7. **JVM Bytecode & Dynamic Proxies:** Contrast JDK Dynamic Proxies with ByteBuddy/CGlib.
8. **Java Collections Internals:** Explain the internal bucket structure, rehash mechanism, and performance characteristics of HashMap.
9. **Lock Free Data Structures:** Explain how 'AtomicReference' uses Compare-And-Swap (CAS) for thread safety.
10. **Custom Annotations & Reflection:** How do retention policies ('RUNTIME' vs 'CLASS') affect reflection-based frameworks?
11. **Java EE Contexts & Dependency Injection (CDI):** How does CDI scope management work? Discuss @Inject and @Managed.
12. **JTA & Distributed Transactions:** Explain the Two-Phase Commit (2PC) protocol and its limitations.
13. **JMS & Message Driven Beans (MDB):** How does message acknowledgement work in JMS? Discuss @MessageDriven.
14. **JPA/Hibernate Caching & Performance:** Explain the N+1 select problem. Compare first-level and second-level caching.
15. **Java Serialization & Vulnerabilities:** Why is native Java serialization considered insecure? Discuss alternatives like Kryo.

```
---
```

```
## 2. Comprehensive Spring Framework (20 Questions)
```

Spring IoC & Core

- 16. ****Bean Lifecycle:**** Trace the detailed lifecycle of a Spring Bean from instantiat
- 17. ****Circular Dependencies:**** How does Spring resolve circular dependencies using th
- 18. ****Custom Spring Starters:**** How do you build a custom Spring Boot Auto-Configurat

Spring MVC & Boot

- 19. ****Request Handling Flow:**** Trace a request from the servlet container through `Di
- 20. ****Spring Boot Startup Mechanics:**** What happens internally when `SpringApplicatio

Spring AOP

- 21. ****Aspect Architecture:**** Explain the difference between compile-time / load-time
- 22. ****Self-Invocation Issue:**** Why doesn't `@Transactional` or `@Cacheable` trigger w

Spring Security

- 23. ****Security Filter Chain:**** Explain the order and execution of filters in `Securit
- 24. ****Authentication Mechanism:**** How does Spring Security authenticate requests usin
- 25. ****OAuth2 / OIDC:**** Describe how Spring Security implements the OAuth2 Authorizati

Spring Data & JDBC

- 26. ****Transaction Propagation:**** Explain the difference between `REQUIRED`, `REQUIRES
- 27. ****Transaction Isolation Levels:**** How do database isolation levels (`READ_COMMITT
- 28. ****Spring Data Custom Repositories:**** How does Spring Data JPA dynamically generat
- 29. ****JdbcTemplate vs JPA:**** When is it architecturally preferable to use `JdbcTempla

Spring Testing

- 30. ****Context Caching:**** How does `@SpringBootTest` cache the ApplicationContext acro
- 31. ****Testing Slice Annotations:**** Compare `@WebMvcTest`, `@DataJpaTest`, and `@JsonT
- 32. ****Testcontainers Integration:**** How do you set up dynamic database containerized
- 33. ****Mocking Static Methods & Private States:**** Why is mocking static methods genera
- 34. ****Transactional Tests:**** Why do `@Transactional` annotations on test methods auto
- 35. ****Spring Security Testing:**** How do you test secured endpoints using `@WithMockUs

3. SQL & Oracle Database Management (15 Questions)

- 36. ****Oracle Indexing Architecture:**** Compare B-Tree, Bitmap, and Function-Based inde
- 37. ****Execution Plan Analysis:**** How do you analyze an Oracle Execution Plan (`EXPLAI
- 38. ****Oracle Partitioning:**** Compare Range, List, Hash, and Composite partitioning. H
- 39. ****Analytic (Window) Functions:**** Explain the difference between `RANK()`, `DENSE_
- 40. ****PL/SQL Subprograms & Packages:**** How do PL/SQL Cursor Variables (`REF CURSOR`),
- 41. ****Locking & Concurrency:**** Explain Oracle's multi-version concurrency control (MV
- 42. ****Flashback & Undo Tablespace:**** How does Oracle use the `UNDO` tablespace to sup
- 43. ****Hierarchical Queries:**** Explain how to write recursive queries using Oracle's `
- 44. ****High Availability (RAC & Data Guard):**** Explain the architecture of Oracle Real
- 45. ****Database Maintenance & Statistics:**** Why is gather table statistics (`DBMS_STAT
- 46. ****MERGE Statements:**** Write and explain an upsert operation using the SQL `MERGE
- 47. ****Temporary Tables & LOB Handling:**** Compare Global Temporary Tables (GTT) with n
- 48. ****Query Optimization:**** How do hints like `/\*+ INDEX(t idx\_name) \*/` or `/*+ LEAD

49. ****Pessimistic vs Optimistic Locking:**** How do you implement pessimistic locking (
50. ****Materialized Views:**** What is the difference between 'ON COMMIT' and 'ON DEMAND

4. Servlets, Tomcat, and WebLogic Environments (15 Questions)

51. ****Servlet Lifecycle:**** Explain 'init()', 'service()', and 'destroy()'. How does a
52. ****Filter vs Servlet vs Listener:**** Compare 'javax/servlet.Filter', 'HttpS
53. ****Async Servlet Processing:**** How does asynchronous servlet processing ('AsyncCon
54. ****Session Management & Clustering:**** How are HTTP Sessions stored, updated, and r
55. ****Tomcat Architecture:**** Explain the hierarchy of Tomcat components: 'Server' ->
56. ****Tomcat Thread Pool Tuning:**** How do parameters like 'maxThreads', 'minSpareThre
57. ****Connectors (NIO vs APR):**** What is the difference between Tomcat's HTTP/NIO, HT
58. ****WebLogic Domain Architecture:**** Explain the roles of the Administration Server,
59. ****WebLogic Work Managers:**** How do WebLogic Work Managers customize thread schedu
60. ****JNDI & DataSource Management:**** How are DataSources and Connection Pools (e.g.,
61. ****Classloading in WebLogic:**** Explain WebLogic's hierarchical classloading mechan
62. ****Shared Libraries & EAR Deployment:**** Compare WAR vs. EAR deployments. How do sh
63. ****Garbage Collection Logs in Containers:**** How do you enable and extract verbose
64. ****WebLogic Security Realms:**** How do WebLogic Security Realms integrate with LDAP
65. ****Graceful Shutdown:**** What happens behind the scenes during a graceful shutdown

5. Linux, Bash, and CI/CD (TeamCity & Jenkins) (20 Questions)

Linux System Administration & Bash

66. ****File Permissions & ACLs:**** Explain standard Linux permissions ('chmod 755', 'ch
67. ****Process & Resource Monitoring:**** How do you troubleshoot high CPU or memory con
68. ****Network Diagnostics:**** How do you check listening ports, open sockets, and netw
69. ****System Logs & Journald:**** How do you inspect and filter system logs using 'jour
70. ****Process & Service Handling:**** Compare Systemd vs SysVinit. How do you create, s
71. ****Security Keys & TLS Certificates:**** How do you inspect, generate, and convert S
72. ****Bash Scripting - Error Handling:**** Write a robust Bash script pattern that uses
73. ****Bash Text Processing:**** Write a command pipeline using 'awk' and 'sed' to extra
74. ****User & Group Administration:**** How do you safely create users, add them to spec
75. ****Storage & Disk Space:**** How do you identify disk space hogs and inode exhaustio

CI/CD Pipelines (TeamCity & Jenkins)

76. ****Jenkins Pipeline Architecture:**** Compare Declarative vs Scripted Jenkins pipeli
77. ****Jenkins Shared Libraries:**** How do you create and use reusable Jenkins Shared L
78. ****TeamCity Build Configurations:**** Explain the concepts of Build Steps, Build Tri
79. ****TeamCity Build Chains:**** How do build chains allow parallel execution, pipeline
80. ****Build Agent Management:**** How do TeamCity Agent Pools and Jenkins Controller-Ag
81. ****Pipeline Security & Secrets Management:**** How do you securely inject credential
82. ****Artifact Management:**** How do CI tools integrate with enterprise artifact repos
83. ****Automated Testing & Quality Gates:**** How do you integrate static analysis tools
84. ****Zero Downtime Deployment Strategies:**** Compare Blue/Green, Canary, and Rolling

85. ****Pipeline Caching:**** How do you configure caching of Maven (`~/m2`) or Gradle d

6. Soft Skills: Analytical Problem-Solving & Team Consensus (10 Questions)

- 86. ****Complex Incident Root Cause Analysis:**** Describe a scenario where a production
- 87. ****Architectural Trade-offs:**** Tell me about a time you had to choose between a qu
- 88. ****Handling Technical Debt:**** How do you identify, prioritize, and articulate the
- 89. ****Building Team Consensus:**** Describe a situation where senior engineers on your
- 90. ****Mentorship & Code Reviews:**** How do you handle code reviews where a peer's solu
- 91. ****Debugging Unknown Systems:**** You are assigned to fix a critical performance bug
- 92. ****Fostering Ownership:**** How do you encourage a team culture where developers tak
- 93. ****Post-Mortem Facilitation:**** Describe your process for running a Blameless Post-
- 94. ****Disagree and Commit:**** Can you share an instance where you strongly disagreed w
- 95. ****Communicating Risk:**** How do you present potential technical risks (such as dat

7. Senior-Level System Design & Architecture Scenarios (5 Questions)

- 96. ****Legacy Migration:**** How would you plan and execute the gradual migration of a m
 - 97. ****High-Throughput Ingestion:**** Design a system capable of handling 50,000 request
 - 98. ****Distributed Transaction Management:**** How do you maintain data consistency acro
 - 99. ****Zero-Downtime Database Schema Migration:**** How do you execute schema alteration
 - 100. ****Resilience & Fault Tolerance:**** How do you design a Spring Boot enterprise app
- ``eof

Summary of Material Provided

I have compiled a comprehensive list of ****100 high-level technical and soft-skills qu**

- 1. ****Java SE & Java EE:**** Advanced concurrency, memory management, JVM internals, vir
- 2. ****Spring Framework:**** Deep dive into IoC, Spring Boot internals, AOP dynamic proxi
- 3. ****SQL & Oracle DB:**** Optimizer execution plans, B-Tree/Bitmap indexes, analytic wi
- 4. ****Servlets, Tomcat & WebLogic:**** Lifecycle execution, container thread pool parame
- 5. ****Linux, Bash & CI/CD:**** Diagnostics, POSIX permissions, log parsing, shell script
- 6. ****Soft Skills & Consensus:**** Scenario questions focusing on problem-solving under
- 7. ****System Design & Architecture:**** Enterprise-scale architectural problems covering

Good luck with your preparation for the interview!

А тепер подай відповіді на кожне запитання
українською мовою у форматі - питання-відповідь.

Ось детальні відповіді на всі 100 запитань для підготовки до співбесіди на позицію **Senior Java Developer**, розподілені за відповідними категоріями.

1. Advanced Java SE & Java EE

1. Поясніть гарантію `happens-before` у Java Memory Model (JMM). Як вона пов'язана з `volatile`, `synchronized` та `final` полями?

`happens-before` визначає порядок, у якому операції запису одного потоку гарантовано стають видимими для читання іншим потоком.

- `volatile`: Запис у `volatile` змінну `happens-before` будь-якого наступного читання цієї ж змінної.
- `synchronized`: Вихід із монітора/блокування `happens-before` входу в цей же монітор.
- `final`: Запис у `final` поле в конструкторі `happens-before` того, як посилання на об'єкт стане видимим для інших потоків (за умови відсутності витоку `this`).

2. Порівняйте архітектурні відмінності та паузи GC між G1GC, ZGC та Shenandoah. Як діагностувати довготривалі паузи?

- **G1GC**: Регіональний збирач сміття. Ділить пам'ять на регіони однакові за розміром, орієнтується на задану користувачем паузу (`MaxGCPauseMillis`). Паузи частіше в межах 10–200 мс.
- **ZGC / Shenandoah**: Низьколатентні збирачі сміття (Concurrent GC). Виконують маркування та переміщення об'єктів паралельно з роботою застосунку (за допомогою Colored Pointers у ZGC та Load Barriers). Забезпечують паузи < 1–10 мс незалежно від розміру хешу (навіть на терабайтах).
- **Діагностика**: Увімкнення GC-логів (`-Xlog:gc*`), аналіз через GCEasy або JDK Mission Control (JFR), та відстеження пауз Stop-The-World (STW).

3. Чим Virtual Threads (Project Loom) відрізняються від platform threads?

Platform threads прямо відповідають потокам ОС (1:1), є дорогими (близько 1 МБ на потік) та обмеженими у кількості (тисячі). Virtual Threads — це легковагові потоки (M:N), які керуються JVM, мають початковий розмір у кілька сотень байт і дозволяють створювати мільйони потоків.

- **Pinning (запинення)**: Це ситуація, коли Virtual Thread прив'язується до Platform Thread під час виконання native-коду або всередині блоку `synchronized`, що заважає JVM звільнити потік ОС при блокуючих операціях.

4. Поясніть правило PECS у Generics. Чому масиви дженериків заборонені та як Type Erasure впливає на Reflection?

- **PECS (Producer Extends, Consumer Super):** Використовуйте `? extends T`, якщо об'єкт лише віддає дані (Producer), і `? super T`, якщо об'єкт лише приймає дані (Consumer).
- **Масиви:** Вони є коваріантними та перевіряють типи під час виконання (reified), а дженерики — інваріантні та стираються під час компіляції. Поєднання призвело б до `ArrayStoreException` під час виконання.
- **Type Erasure & Reflection:** Стирання типів видаляє інформацію про дженерики у байт-кодi. Проте інформація про типи зберігається у сигнатурах класів/методів і доступна через `GenericDeclaration` у Reflection.

5. Порівняйте `CompletableFuture` та `Reactive Streams (Project Reactor)`. Як обробляти винятки через потоки?

`CompletableFuture` підходить для асинхронних задач за принципом Push (одноразовий результат або ланцюг задач). `Reactive Streams (Mono/Flux)` реалізують модель Push-Pull з підтримкою Backpressure (регулювання навантаження джерелом).

- **Винятки:** У `CompletableFuture` використовують `.exceptionally()` або `.handle()`. У Reactor — `.onErrorResume()`, `.onErrorReturn()` або `.retry()`.

6. Поясніть ієрархію `ClassLoader`. Як вони викликають `memory leaks` і що змінили модулі (JPMS)?

- **Ієрархія:** `Bootstrap ClassLoader` -> `Platform ClassLoader` -> `Application ClassLoader`. Працює за принципом делегування наверх.
- **Memory Leaks:** Якщо кастомний `ClassLoader` (наприклад, у веб-контейнері при redeploy) завантажує клас, а посилання на екземпляр або `ThreadLocal` залишається у системному потоці, `ClassLoader` не може бути зібраний GC разом із усіма його класами (PermGen/Metaspace leak).
- **JPMS (Java 9):** Ввів жорсткі межі інкапсуляції (Modules), контролюючи експорт пакетів і забороняючи доступ до внутрішніх API за замовчуванням.

7. Контраст між `JDK Dynamic Proxies` та `CGLIB/ByteBuddy`. Як вони працюють?

- **JDK Dynamic Proxy:** Працює лише через інтерфейси (`java.lang.reflect.Proxy`), створює проксі-клас у пам'яті за допомогою Reflection.
- **CGLIB / ByteBuddy:** Генерує підклас (subclass) цільового класу через пряму маніпуляцію байт-кодом. Не працює з `final` класами та `final` методами.

8. Внутрішня структура `HashMap` у Java 8+. Що викликає проблеми конкаренсі?

- **Структура:** Масив бакетів (`Node<K,V>[]`). При колізіях елементи зв'язуються у зв'язаний список. Якщо кількість елементів у бакеті перевищує 8 (і загальна

кількість бакетів ≥ 64), список перетворюється на Червоно-чорне дерево ($O(\log N)$).

- **Concurrency:** Небезпека паралельного модифікування: втрата оновлень (lost updates), неатомний `resize()`, що в старій Java 7 призводив до зациклення зациклених списків.

9. Як `AtomicReference` використовує CAS? Що таке ABA проблема і як її вирішує `AtomicStampedReference`?

- **CAS (Compare-And-Swap):** Атомарна інструкція процесора, яка змінює значення, тільки якщо воно відповідає очікуваному.
- **ABA проблема:** Потік A бачить значення A, потік B змінює A $\rightarrow$ B $\rightarrow$ A. Потік A виконує CAS, вважаючи, що стан не змінювався, хоча контекст змінився.
- **Вирішення:** `AtomicStampedReference` додає версійність (мітку/штамп цілого числа) до посилання.

10. Області видимості анотацій (`RUNTIME`, `CLASS`, `SOURCE`). Наслідки Reflection для продуктивності?

- **SOURCE:** Видаляються компілятором (наприклад, `@Override`, Lombok).
- **CLASS:** Зберігаються у `.class` файлі, але недоступні у runtime (використовуються інструментами аналізу).
- **RUNTIME:** Зберігаються у байт-кодi та доступні під час виконання через Reflection.
- **Продуктивність:** Reflection вимагає додаткових перевірок доступу та метаданих, що вимикає JIT-оптимізації (inline). Для пом'якшення застосовують кешування об'єктів `Method` / `Field` або `MethodHandles`.

11. Як CDI scopes у Java EE відрізняються від Spring IoC bean scopes?

В CDI проксі-об'єкти створюються за замовчуванням для багатьох scopes (`@RequestScoped`, `@SessionScoped`), і виклик методу контекстного сфера-об'єкта завжди перехоплюється проксі-клієнтом. У Spring Singleton конструюється одразу, а для короткоживучих бинів (`@RequestScope`) у singleton-послугах потрібні explicit `proxyMode = ScopedProxyMode.TARGET_CLASS`.

12. Поясніть двофазний коміт (2PC) у JTA та поведінку при розриві мережі.

- **Фази:** 1) *Prepare phase* (координатор питає всі ресурси, чи готові вони фіксувати); 2) *Commit/Rollback phase* (якщо всі відповіли "так", відправляється команда зафіксувати).
- **Network Partition:** Якщо мережа рветься під час другої фази, ресурси залишаються у підвишеному стані (блокують локальні транзакційні записи) до відновлення зв'язку або ручного втручання адміністратора (heuristic completion).

13. Підтвердження повідомлень у JMS (Message Driven Beans).

- **AUTO_ACKNOWLEDGE:** Контейнер/клієнт автоматично підтверджує отримання після успішного повернення з методу.
- **CLIENT_ACKNOWLEDGE:** Застосунок повинен явно викликати `message.acknowledge()`.
- **У MDB (EJB):** Підтвердження контролюється транзакцією контейнера (CMT). Якщо транзакція відкочується, повідомлення повертається у чергу (redelivery).

14. Проблема N+1 у JPA/Hibernate. Рішення.

- **Проблема:** При вибірці N батьківських сутностей виконання N додаткових запитів для завантаження кожної пов'язаної Child-сутності.
- **Рішення:** `JOIN FETCH` у JPQL/HQL, `@EntityGraph`, виклики Batch Fetching (`@BatchSize(size = 20)`).

15. Чому нативна серіалізація небезпечна?

Під час десеріалізації об'єкту виконується байт-код без виклику конструктора, що дозволяє зловмисникам конструювати "gadget chains" (ланцюги викликів `readObject()`) для виконання довільного коду (RCE).

- **Захист:** Використання форматів JSON/Protobuf, обмеження через `ObjectInputFilter`.

2. Comprehensive Spring Framework

16. Повний життєвий цикл Spring Bean.

1. Сканування та визначення (`BeanDefinition`).
2. Інстанціювання об'єкта.
3. Впровадження залежностей (DI).
4. Виклики Aware-інтерфейсів (`BeanNameAware`, `ApplicationContextAware`).
5. `BeanPostProcessor.postProcessBeforeInitialization()`.
6. `@PostConstruct` / `InitializingBean.afterPropertiesSet()` / init-method.
7. `BeanPostProcessor.postProcessAfterInitialization()` (тут створюються AOP проксі).
8. Бин готовий до використання.
9. `@PreDestroy` / `DisposableBean.destroy()` / destroy-method при закритті контексту.

17. Циклічні залежності та трирівневий кеш Spring.

Spring вирішує циклічні залежності для singleton-бинів через виклики сетерів за допомогою 3 кешів:

1. `singletonObjects` (готові бини).
 2. `earlySingletonObjects` (ранні посилання на бини до ініціалізації).
 3. `singletonFactories` (фабрики `ObjectFactory` для створення проксі).
- **Падіння:** Якщо залежність через конструктори, Spring не може створити екземпляр і викидає `BeanCurrentlyInCreationException`. Також падає при використанні `@Async` на циклічних бинах.

18. Створення кастомного Spring Boot Auto-Configuration.

1. Створити `@Configuration` клас із необхідними бинами.
2. Використати умовні анотації (`@ConditionalOnProperty` , `@ConditionalOnClass` , `@ConditionalOnMissingBean`).
3. Зареєструвати клас у файлі `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` .

19. Потік обробки запиту в Spring MVC.

`HttpServletRequest` -> `DispatcherServlet` -> `HandlerMapping` (визначення контролера) -> `HandlerAdapter` (викликає `HandlerInterceptor.preHandle()`) -> Виконання `@Controller` / `@RestController` -> `HandlerInterceptor.postHandle()` -> Обробка винятків у `@ControllerAdvice` -> Повернення `ResponseEntity` або `ModelAndView` .

20. Внутрішній механізм `SpringApplication.run()` .

Запускає секундомір, створює `BootstrapContext` , визначає тип застосунку (REACTIVE / SERVLET), шукає та запускає `SpringApplicationRunListeners` , створює `ApplicationContext` , завантажує джерела конфігурації (включаючи Auto-configurations), викликає `refresh()` , запускає `CommandLineRunner` та `ApplicationRunner` .

21. AspectJ vs Dynamic Proxy у Spring AOP.

- **Spring AOP (Dynamic Proxy):** Працює під час виконання (runtime). Використовує JDK Proxies або CGLIB. Перехоплює лише публічні виклики методів через Spring-бини.
- **AspectJ:** Повноцінний фреймворк AOP. Впроваджує аспекти на етапі компіляції (CTW) або завантаження класів (LTW). Може перехоплювати приватні методи, виклики конструкторів, стативні методи.

22. Проблема внутрішнього виклику методів (Self-Invocation) в AOP.

Оскільки Spring AOP огортає бин у проксі, виклик методу *A* з методу *B* усередині того ж об'єкта (`this.methodA()`) оминає проксі.

- Рішення: Винесення методу в інший Spring-бин, самоін'єкція (`@Autowired` на самого себе), або використання `AopContext.currentProxy()` .

23. Архітектура Spring Security Filter Chain.

Spring Security додає єдиний `DelegatingFilterProxy` до Servlet Container Filter Chain. Цей проксі делегує обробку Spring-бину `FilterChainProxy` , який містить список `SecurityFilterChain` (фільтри на кшталт `UsernamePasswordAuthenticationFilter` , `BearerTokenAuthenticationFilter` , `FilterSecurityInterceptor`).

24. Механізм аутентифікації у Spring Security.

Запит -> `AuthenticationFilter` створює неаутентифікований `Authentication` токен -> пересилає в `AuthenticationManager` (`ProviderManager`) -> опитуються `AuthenticationProvider` -> підтягуються деталі через `UserDetailsService` -> повертається аутентифікований `Authentication` об'єкт -> зберігається у `SecurityContextHolder.getContext().setAuthentication(...)` .

25. OAuth2 Authorization Code Grant з PKCE у Spring Security.

Клієнт генерує `code_verifier` та його хеш `code_challenge` . Клієнт перенаправляє користувача на Authorization Server з `code_challenge` . Після входу сервер повертає `authorization_code` . Клієнт обмінює `authorization_code` + оригінальний `code_verifier` на `access_token` . PKCE запобігає перехопленню коду авторизації.

26. Рівні розповсюдження транзакцій (`Propagation`).

- `REQUIRED` : Використовує поточну транзакцію, створює нову, якщо відсутня.
- `REQUIRES_NEW` : Завжди створює нову транзакцію, призупиняючи поточну.
- `NESTED` : Створює savepoint у межах поточної транзакції.
- `MANDATORY` : Вимагає наявності поточної транзакції, інакше викидає виняток.

27. Рівні ізоляції транзакцій у Spring.

Мапуються на рівні DB: `READ_UNCOMMITTED` , `READ_COMMITTED` , `REPEATABLE_READ` , `SERIALIZABLE` . Phantom reads (коли повторний вибірковий запит повертає нові рядки, додані іншою транзакцією) запобігають за допомогою рівнів `SERIALIZABLE` або `REPEATABLE_READ` (у залежності від реалізації MVCC в БД).

28. Як працює Spring Data JPA під капотом?

Spring генерує динамічний проксі для інтерфейсу репозиторію під час старту контексту за допомогою `RepositoryFactorySupport` . Проксі аналізує імена методів (Query DSL) або анотації `@Query` та транслює їх у JPA/HQL запити.

29. `JdbcTemplate` проти JPA/Hibernate.

`JdbcTemplate` краще використовувати при потребі у максимальній продуктивності, складних аналітичних SQL-запитах, масових вставках (Bulk inserts) або використанні

специфічних фіч БД. JPA зручніший для стандартного CRUD, графу об'єктів та доменної логіки.

30. Кешування контексту у `@SpringBootTest`.

Spring TestContext Framework кешує `ApplicationContext` між тестами за їх конфігурацією. Якщо тест змінює стан контексту (наприклад, перевизначає бин), виклик `@DirtiesContext` примушує розробників перезавантажувати контекст, що істотно уповільнює тести.

31. Slice Annotations у Spring Testing.

- `@WebMvcTest` : Завантажує лише веб-шар (Controllers, Interceptors, Spring Security).
- `@DataJpaTest` : Завантажує лише JPA сутності, репозиторії та налаштовує embedded DB.
- `@MockBean` : Замінює існуючий бин або додає мок у контекст. Може призвести до скидання кешу контексту, якщо конфігурація тестів відрізняється.

32. Інтеграція Testcontainers у Spring Boot 3.

Використовується `@Testcontainers` та `@Container` для старту Docker-контейнерів (наприклад, Oracle/PostgreSQL). За допомогою `@DynamicPropertySource` параметри підключення (URL, login, password) динамічно підставляються в `Environment` Spring.

33. Мокування зовнішніх HTTP-залежностей.

Мокування статичних методів є симптомом поганого дизайну. Для тестування HTTP-клієнтів використовується WireMock або `MockRestServiceServer`, які піднімають локальний HTTP-сервер для перевірки мережових викликів.

34. Автоматичний rollback у транзакційних тестах.

Тести з `@Transactional` за замовчуванням виконують rollback після кожного тесту для чистоти даних. Пастка: `LazyInitializationException` не виникає під час тесту, оскільки сесія Hibernate залишається відкритою протягом всього тесту, але виникне в продакшені.

35. Тестування захищених ендпоінтів.

Використовують `@WithMockUser(username="admin", roles={"ADMIN"})` або `SecurityMockMvcRequestPostProcessors.jwt()` для створення тестового `SecurityContext`.

3. SQL & Oracle Database Management

36. Індокси у Oracle (B-Tree, Bitmap, Function-Based).

- **B-Tree**: Стандартний індекс для даних із високою селективністю (багато унікальних значень).

- **Bitmap:** Для даних із низькою селективністю (наприклад, стать, статус). Використовує бітові карти. Вразливий до блокувань при частих UPDATE.
- **Function-Based:** Індує результат функції (наприклад, `UPPER(column)`).
- **Пропуск індексу CBO:** Виникає, якщо селективність низька (простіше зробити Full Table Scan), при використанні операторів `!=`, `NOT IN`, або застосуванні функцій до колонки без Function-Based індексу.

37. Аналіз Execution Plan в Oracle.

- `TABLE ACCESS FULL` : Повне сканування таблиці.
- `INDEX RANGE SCAN` : Вибір за діапазоном індексу.
- `NESTED LOOPS` : Ефективне з'єднання для малих наборів даних.
- `HASH JOIN` : Ефективне з'єднання для великих наборів даних.

38. Партиціонування в Oracle.

Ділить великі таблиці на менші частини (Range, List, Hash, Composite). **Partition Pruning:** Оптимізатор Oracle автоматично відкидає партиції, які не відповідають умовам запиту `WHERE`, зменшуючи обсяг зчитуваних даних.

39. Аналітичні (віконні) функції.

`RANK()` залишає пропуски у нумерації при дублікатах (1, 2, 2, 4), `DENSE_RANK()` не залишає пропусків (1, 2, 2, 3), `ROW_NUMBER()` повертає унікальний номер кожного рядка (1, 2, 3, 4).

40. PL/SQL Bulk Collect та FORALL.

Мінімізують переключення контексту (Context Switches) між PL/SQL та SQL-рушієм. `BULK COLLECT` зчитує вибірку в колекцію за один виклик, а `FORALL` виконує масові DML-операції пакетом.

41. MVCC у Oracle та ORA-00060.

Oracle реалізує MVCC за допомогою `UNDO` блоків. Запити на читання створюють узгоджене за часом відображення даних без блокувань читання. `ORA-00060: deadlock detected` виникає, коли два сеанси очікують на ресурси, заблоковані один одним.

42. Flashback та UNDO tablespace.

Oracle зберігає старі версії блоків у `UNDO` сегментах. Це дозволяє виконувати `SELECT * FROM table AS OF TIMESTAMP...` (Flashback Query) для відновлення випадково видалених даних.

43. Ієрархічні запити.

Використовують `CONNECT BY PRIOR parent_id = child_id START WITH parent_id IS NULL` або стандартизовані recursive CTE (`WITH r AS (...)`).

44. High Availability (RAC & Data Guard).

- **RAC (Real Application Clusters):** Кілька вузлів працюють з єдиною БД (Shared Storage).
- **Data Guard:** Основна БД передає REDO логи на резервну (Standby).
- **FCF (Fast Connection Failover):** Дозволяє JDBC драйверу перепідключатися до живого вузла без падіння застосунку.

45. DBMS_STATS.GATHER_TABLE_STATS .

Збирає статистику щодо розподілу даних (гістограми, кількість рядків). Застаріла статистика змушує СВО будувати неефективні планом виконання запитів.

46. Синтаксис MERGE INTO .

SQL



```

MERGE INTO target_table t
USING source_table s ON (t.id = s.id)
WHEN MATCHED THEN UPDATE SET t.val = s.val
WHEN NOT MATCHED THEN INSERT (id, val) VALUES (s.id, s.val);

```

47. Обробка LOB (CLOB/BLOB) у Java.

Не завантажуйте LOB у пам'ять цілком як String чи byte[]. Використовуйте потокову обробку (InputStream / Reader) через JDBC driver або LOB-потоки JPA.

48. Підказки оптимізатора (Hints).

Дозволяють явно вказати СВО спосіб виконання (/*+ INDEX(...) */ , /*+ USE_HASH(...) */). Ризик: зі зміною обсягу даних або версії БД підказка може погіршити продуктивність.

49. Pessimistic Locking у Spring/Oracle.

Запит SELECT ... FOR UPDATE NOWAIT викликає блокування рядка. У Spring Data це @Lock(LockModeType.PESSIMISTIC_WRITE). Якщо рядок заблоковано, Oracle кидає ORA-00054, що перетворюється у PessimisticLockingFailureException.

50. Materialized Views (MView).

Кешують результат запиту на диску. ON COMMIT оновлює MView при комміті в базовій таблиці, ON DEMAND — за розкладом чи вручну. Fast Refresh оновлює лише зміни за допомогою MView Logs.

4. Servlets, Tomcat, and WebLogic Environments

51. Життєвий цикл сервлета.

init() (одноразово) -> service() (викликається у кожному потоці для запиту) ->

`destroy()` (при зупинці). Servlet є Singleton, тому сервлет-контейнер обробляє запити паралельно; розробник повинен уникати стану (state) у полях класу сервлета.

52. Filter vs Servlet vs Listener.

- `Listener` : Реагує на події контейнера (створення сесії, запуск контексту).
- `Filter` : Перехоплює запити ДО досягнення сервлета та відповіді ПІСЛЯ (для логування, авторизації).
- `Servlet` : Основний компонент обробки бізнес-логіки.

53. Asynchronous Servlet Processing.

Метод `request.startAsync()` звільняє потік контейнера для обробки нових HTTP-запитів, передаючи обробку довготривалої задачі (I/O, виклики API) в окремий потік застосунку через `AsyncContext`.

54. Кластеризація та сесії.

HTTP-сесії можуть копіюватися між вузлами кластера (Session Replication) або використовується sticky sessions на рівні Load Balancer (запит завжди йде на той самий вузол).

55. Ієрархія компонентів Tomcat.

`Server` (контейнер вищого рівня) -> `Service` -> `Connector` (обробляє мережеві з'єднання) -> `Engine` -> `Host` (віртуальний хост) -> `Context` (веб-застосунок).

56. Налаштування thread pool Tomcat.

- `maxThreads` : Максимальна кількість робочих потоків (за замовчуванням 200).
- `minSpareThreads` : Мінімальна кількість активних потоків.
- `acceptCount` : Довжина черги операційної системи при вичерпанні всіх `maxThreads`.

57. Connectors у Tomcat (NIO vs APR).

- `NIO` : Неблокуючий I/O на чистому Java API.
- `APR (Apache Portable Runtime)` : Використовує нативні бібліотеки C та OpenSSL для підвищення продуктивності мережі та шифрування.

58. Доменна архітектура WebLogic.

- `Admin Server` : Центральний вузол для керування конфігураціями.
- `Managed Servers` : Вузли, на яких розгортаються застосунки.
- `Node Manager` : Процес ОС для дистанційного запуску/зупинки Managed Servers.

59. WebLogic Work Managers.

Замість стандартних потоків WebLogic керує потоками через Work Managers на основі політик: Fair Share (рівномірне виділення ресурсів), Response Time (пріоритет часу відповіді) та Capacity Constraints.

60. Connection Pools (HikariCP / WebLogic GridLink).

Контролюють створення, повторне використання та перевірку ("validation query") підключень до БД. GridLink в WebLogic інтегрований з Oracle RAC для миттєвого відпрацювання збоїв (Fast Application Notification).

61. Classloading у WebLogic.

Ієрархічний. `weblogic-application.xml` з директивою `<prefer-application-packages>` змушує WebLogic використовувати версію бібліотеки з WAR/EAR файлу, а не з вбудованих системних бібліотек WebLogic.

62. WAR vs EAR та Shared Libraries.

- **WAR:** Модуль веб-застосунку.
- **EAR:** Архів, що включає кілька WAR та EJB модулів.
- **Shared Libraries:** Дозволяють розгортати спільні JAR-файли в контейнер WebLogic один раз і посилатися на них із різних EAR/WAR через `MANIFEST.MF`.

63. Вилучення Thread Dumps та GC логів у проді.

Використовують CLI інструменти: `jstack <pid>`, `jcmd <pid> Thread.print`, або `kill -3 <pid>` у Linux для запису Thread Dump у log.

64. WebLogic Security Realms.

Компоненти WebLogic для аутентифікації та авторизації. Дозволяють підключати зовнішні провайдери (LDAP, Active Directory) без зміни коду застосунку.

65. Graceful Shutdown.

Контейнер зупиняє прийом нових з'єднань, чекає виконання поточних HTTP-запитів протягом тайм-ауту (`shutdown timeout`), закриває сесії, зупиняє Spring Context та завершує процес.

5. Linux, Bash, and CI/CD (TeamCity & Jenkins)

66. Права доступу в Linux.

- `chmod 755` : `rw-r-xr-x` (власник — все, група і інші — читання/виконання).
- `SUID / SGID` : Дозволяє виконувати файл із правами власника/групи файлу.
- `Sticky Bit` : Не дозволяє видаляти файли у каталозі не-власникам (наприклад, `/tmp`).
- `ACLs` : Розширені права (`setfacl -m u:user:rw file`).

67. Моніторинг ресурсів у Linux.

- `top` / `htop` : Загальний стан системних ресурсів і процесів.
- `pidstat -t -p <pid>` : CPU/пам'ять по потоках конкретного Java-процесу.
- `free -m` : Використання RAM і SWAP.

68. Мережева діагностика.

- `ss -tulpn` або `netstat -tulpn` : Порти та процеси, які їх слухають.
- `lsof -i :8080` : Відкриті сокети на порту 8080.
- `nc -zv <host> <port>` : Перевірка доступності порту на віддаленому хості.

69. Пошук та аналіз логів.

`journalctl -u my-app.service -n 100 --no-pager` або робота з файлами: `grep -i "ERROR" app.log | awk '{print $1, $2, $5}' | sort | uniq -c`.

70. Створення Systemd сервісу.

Створюють файл `/etc/systemd/system/my-app.service` :

Ini, TOML



```
[Unit]
Description=Java App
After=network.target
[Service]
User=appuser
ExecStart=/usr/bin/java -jar /opt/app/app.jar
Restart=always
[Install]
WantedBy=multi-user.target
```

Запуск: `systemctl daemon-reload && systemctl enable --now my-app`.

71. Сертифікати SSL/TLS та KeyStore.

`openssl x509 -in cert.crt -text -noout` (перегляд сертифіката).

Додавання сертифіката в Java TrustStore:

`keytool -importcert -alias mycert -file cert.crt -keystore $JAVA_HOME/lib/security/cacerts`.

72. Шаблон надійного Bash-скрипту.

Bash



```
#!/usr/bin/env bash
set -euo pipefail
trap 'echo "Error occurred at line $LINENO"; cleanup' ERR
cleanup() { rm -f /tmp/temp_data; }
```

73. Обробка тексту в Bash.

`awk '/ERROR/ {print $4}' access.log | sort | uniq -c | sort -nr | head -n 5` — знаходить топ 5 частих помилок.

74. Користувачі та SSH.

Створення: `useradd -m -s /bin/bash appuser`. SSH ключі додаються у `/home/appuser/.ssh/authorized_keys` з правами `600`.

75. Дисковий простір.

`df -h` (вільне місце на файлових системах), `df -i` (перевірка вільних inodes), `du -sh /* | sort -h` (розмір каталогів).

76. Jenkins Declarative Pipeline.

Groovy



```
pipeline {
    agent any
    stages {
        stage('Build') { steps { sh 'mvn clean package' } }
        stage('Test') { steps { sh 'mvn test' } }
    }
    post { always { junit '**/target/surefire-reports/*.xml' } }
}
```

77. Jenkins Shared Libraries.

Спільні бібліотеки на Groovy, які зберігаються в окремому Git-репозиторії. Дозволяють перевикористовувати логіку збірки (`vars/myCustomStep.groovy`) у багатьох проектних `Jenkinsfile`.

78. Концепції TeamCity.

- `Build Configuration`: Набір кроків для створення та тестування.

- **Artifact Dependency** : Використання результатів (JAR/WAR) з попередньої конфігурації.
- **Snapshot Dependency** : Гарантує, що всі збірки в ланцюжку виконуються на одному commit із VCS.

79. TeamCity Build Chains.

Граф залежностей збірок (DAG). Дозволяє паралельно виконувати незалежні модулі (наприклад, окремо Unit-тести, UI-тести, статичний аналіз) та об'єднувати їх перед деплоєм.

80. Управління Build Agents.

Агенти — це сервери, які виконують збірку. В Jenkins (Controller-Agent) та TeamCity (Agent Pools) конфігуруються теги/сбірочні можливості (capabilities, наприклад, наявність Docker або JDK 21).

81. Безпека секретів у CI/CD.

Секрети зберігають у HashiCorp Vault, Jenkins Credentials Manager або TeamCity Encrypted Parameters. Не виводять їх у консоль та використовують автоматичне маскування в логах.

82. Інтеграція з Artifact Repositories.

Використання плагінів Nexus/Artifactory для публікації релізів та SNAPSHOT-версій за допомогою Maven (`mvn deploy`) або створюваних Docker-образів.

83. Quality Gates (SonarQube/JaCoCo).

У CI додається крок аналізу коду. За допомогою SonarQube Quality Gate пайплайн падає, якщо відсоток покриття тестами (JaCoCo) нижчий за поріг (наприклад, 80%) або виявлені критичні вразливості (Blocker bugs).

84. Стратегії Zero Downtime Deployment.

- **Blue/Green**: Дві ідентичні середовища; трафік перемикається Load Balancer-ом з Blue на Green.
- **Canary**: Оновлення розгортається на малу частку серверів (5%), перевіряються метрики, після чого оновлюється решта.
- **Rolling**: Послідовна заміна вузлів у кластері по одному.

85. Кешування у CI/CD.

Збереження каталогу `~/.m2/repository` між збірками на агентах для запобігання постійного завантаження всіх залежностей із мережі.

6. Soft Skills: Problem-Solving & Team Consensus

86. Сценарій Root Cause Analysis (RCA).

1. Збір метрик та фактів (метрики Prometheus, лог-файли, thread dumps, heap dump).
2. Формування гіпотези (наприклад, витік пам'яті через неочищений `ThreadLocal`).
3. Відтворення проблеми на Staging середовищі.
4. Розробка та тестування фіксу.
5. Написання Blameless Post-Mortem.

87. Компроміс між Quick Fix та Maintainable Solution.

Визначаю бізнес-пріоритет. Якщо це інцидент критичного рівня (P1), застосовується швидкий тимчасовий фік (Workaround) для відновлення сервісу. Однією з умов фіксу є створення техборгу у бдклові для повноцінного архітектурного рішення.

88. Управління технічним боргом.

Трансляція техборгу мовою бізнесу: оцінка ризиків простою системи, вплив на швидкість розробки фіч (Time-to-Market). Забезпечення "правила скаута" (залишай код чистішим ніж він був до тебе) та виділення ~15-20% часу спринту на рефакторинг.

89. Досягнення консенсусу в команді.

Організація архітектурної сесії (Architecture Decision Record — ADR). Порівняння варіантів за чіткими критеріями: складність підтримки, продуктивність, ризики, час розробки. Якщо точки зору рівнозначні, розробка Proof of Concept (PoC) для об'єктивного вибору.

90. Надання фідбеку під час Code Review.

Фокус на коді, а не на особистості. Використання запитальної форми ("Що ти думаєш про винесення цієї логіки у...?") замість вимог. Визначення критичності зауважень (наприклад, додавання префіксів `NIT:` — некритична порада, або `BLOCKER:` — критична помилка).

91. Робота з невідомим легасі-кодом.

Аналіз за допомогою integration/end-to-end тестів (створення "безпечної сітки" тестів). Використання статичних аналізаторів, побудова діаграм послідовності (Sequence Diagrams) та покрокове логування Execution flow.

92. Культура колективної відповідальності.

Відмова від пошуку винних. Впровадження автоматизованих Quality Gates, сповіщень у чати про статус збірок, чергування на підтримці (On-Call) між усіма Senior/Middle розробниками.

93. Проведення Blameless Post-Mortem.

Аналіз ситуації за методологією "5 Whys" (5 Чому). Мета — знайти прогалини в

процесах, тестуванні або моніторингу, які допустили виникнення помилки, а не покарати людину.

94. Принцип "Disagree and Commit".

Аргументований виклад позиції на етапі обговорення. Якщо рішення прийнято командно або лідом не на користь моєї ідеї — повна підтримка виконання прийнятого рішення без саботажу.

95. Комунікація технічних ризиків.

Опис ризиків у вигляді сценаріїв: "Якщо ми не оновимо версію БД, під час високого навантаження на Чорну П'ятницю сервіс втратить можливість обробляти замовлення, що коштуватиме \$X коштів".

7. System Design & Architecture Scenarios

96. Міграція моноліту на мікросервіси (Strangler Fig Pattern).

1. Виділення кордонів контекстів (Bounded Contexts).
2. Налаштування API Gateway перед монолітом.
3. Покрокове винесення окремих модулів у мікросервіси Spring Boot.
4. Перенаправлення трафіку з моноліту на нові сервіси на рівні Gateway.
5. Повне відключення старого модуля в моноліті.

97. Дизайн системи на 50,000 req/sec.

- **API / Web:** Spring Boot (Virtual Threads або WebFlux) за горизонтально масштабованими Load Balancers (NGINX/HAProxy).
- **Caching:** Багаторівневе кешування (In-Memory Caffeine + Redis Cluster) для обробки sub-millisecond вибірок.
- **Ingestion:** Черги повідомлень (Apache Kafka) для асинхронного буферизованого запису.
- **Storage:** Oracle DB з партиціонуванням, CDC (Debezium) для індексації даних в Elasticsearch/OpenSearch.

98. Распределені транзакції без 2PC (Saga Pattern).

Використання **Saga Pattern** (Choreography або Orchestration). Кожен сервіс виконує локальну транзакцію та публікує подію. У разі помилки виконуються компенсаційні транзакції (Compensating Transactions) для повернення системи у узгоджений стан. Події зберігаються через **Transactional Outbox Pattern**.

99. Схема-міграція бази даних без Downtime.

Застосування підходу **Expand and Contract**:

1. *Expand:* Додати нову колонку у схему БД.

2. *Write to both*: Застосунок пише одночасно у стару та нову колонки.
3. *Backfill*: Скриптом перенести старі дані у нову колонку.
4. *Read*: Переключити читання на нову колонку.
5. *Contract*: Видалити стару колонку. Інструменти: Flyway / Liquibase.

100. Захист від каскадних збоїв (Resilience4j).

- **Circuit Breaker**: Відкриває ланцюг при перевищенні відсотка помилок, припиняючи виклики падаючого сервісу.
- **Bulkhead**: Обмежує кількість паралельних викликів до конкретного ресурсу (ізоляція потоків).
- **Rate Limiter**: Обмежує кількість запитів за проміжок часу.
- **Retry + Fallback**: Автоматичний повтор запиту з експоненційною затримкою та надання дефолтної відповіді при остаточному збої.